

A program célja:

Ez a program egy határidőnapló, amely lehetővé teszi a felhasználó számára az események pontos rögzítését . Az események adatokat (dátum, időpont, helyszín, név, megjegyzés) tartalmaznak, ezen felül tartalmazhatnak további tulajdonságokat(pl: gyakoriság,URL, Prioritás) és rekordokban tárolódnak. A program alapvető célja, hogy egyértelmű és rendezett módon tegye lehetővé az események kezelését és keresését.

Az adatmodell:

- Dátum: ÉÉÉÉ-HH-NN formátumban, pl: "2025-11-08".
- Időpont: OO:PP (24 órás időformátumban) pl: "18:30".
- Esemény neve: Szabadon kitölthető szöveg ,DE kötelezően kitöltendő!!!
- Helyszín: Szabadon kitölthető szöveg, opcionálisan kitöltendő.
- Megjegyzés: Szabadon kitölthető szöveg, opcionálisan kitöltendő.

További adatmodellek:

- URL: https/ Szabadon kitölthető szöveg, opcionálisan kitöltendő.
- Gyakoriság: napi/heti/havi, opcionálisan kitöltendő.
- Prioritás: magas/alacsony, opcionálisan kitöltendő.

A program használata :

A program menüvezérelt felhasználói felületet biztosít, amelyen keresztül a felhasználó különböző funkciókat érhet el:

- Killistázás
- Új esemény (rekord) hozzáadása.
- Esemény módosítása
- Esemény törlése.
- Esemény keresése név alapján.
- Események mentédse adatbázis fájlba

Kilistázás:

Kiírja a kimenetre a listában tárolt adatokat.

Új esemény (rekord) hozzáadása:

A program lépésenként bekéri az esemény adatait(dátum, időpont, esemény neve, helyszín, megjegyzés):

Majd a rendszer ellenőrzi hogy az adatok helyes formátumban vannak-e megadva.

Esemény törlése:

A program egyedi azonosító (dátum és név) alapján keresi meg a törölni kívánt eseményt.A törlést nem lehet utólag módosítani véglegesen megszűnik ez a rekord.

Esemény keresése név alapján:

A program a megadott kulcsszót tartalmazó összes eseményt megjeleníti.

Események mentése adatbázis fájlba:

A program bekér a felhasználótól egy fájl nevet majd létrehoz azzal a fájl névvel egy fájlt, amely tartalmazza az összes jelenlegi rekordot.

Bemeneti/Kimeneti formátumok:

A felhasználó által megadott adatokat a program bemeneti fájlból vagy interaktívan a menüből olvassa be.

Ha a bemeneti fájl egyes soraiban nincsenek meg a kötelezően kitöltendő adattagok (pl dátum,nev) nem olvassa be a program.

Az adatok kimenete szabványos formátumban történik, és csv tehát adatbázis fájlba irányítható.

Az adatok (;)-vel vannak elválasztva (pl: Datum;Idopont;Esemeny neve;Helyszin;Megjegyzes) bemeneten is és a kimeneten is így vannak tagolva.

Hibakezelés:

Helytelen adatbevitel esetén figyelmeztetés és új adatkérelem.(pl:hibás dátum formátum)

Hiányzó kötelező mezők esetén a program nem engedi a mentést.

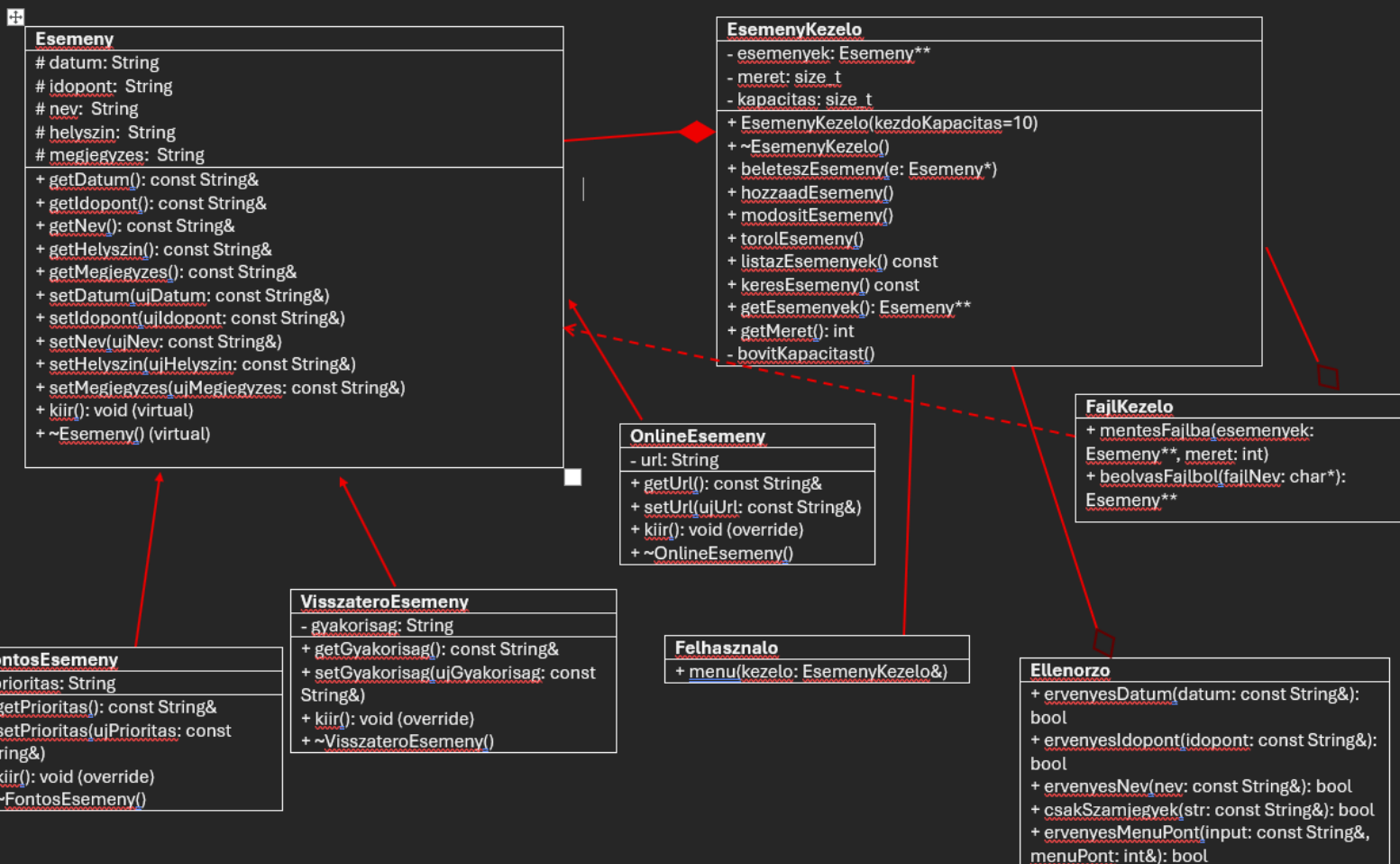
Nem létező esemény keresése vagy törlése esetén hibaüzenet.

A futás eredménye:

A program betölti a főmenüt kiválasztva a megfelelő műveletet, futás során minden funkció azonnali visszajelzést ad és a kiválasztott műveletek végrehajtása után físzajelzéseket biztosít amíg ki nem lépünk a programból.

Segítőábra:

Mező neve	Kötelező ?	Formátum	Leírás
Dátum	Igen	ÉÉÉÉ-HH- NN (pl.: 2025-11-08)	Az esemény dátuma
Időpont	Igen	OO:PP (24 órás formátum, pl.: 18:30)	Az esemény időpontja
Esemény neve	Igen	Szabad szöveg	Az esemény neve (egyedi azonosító a dátummal együtt)
Helyszín	Nem	Szabad szöveg	Az esemény helyszíne (opcionális)
Megjegyzés	Nem	Szabad szöveg	Kiegészítő információ az eseményhez
URL	Nem	Érvényes webcím	Kapcsolódó link, ha szükséges
Gyakoriság	Nem	napi/heti/havi	Az esemény ismétlődése (ha van)
Prioritás	Nem	magas/alacsony	Az esemény fontossági szintje



Programozói dokumentáció:

A program célja:

A célunk hogy a program egy könnyenkezelhető határidő naplót adjon vissza. Ami lehetővé teszi az ujadatok létrehozását, módosítást, torlést, keresést és kilistázást ,adatok csv fájlba mentését a hibákat kiírja és a felhasználó számára javítható legyen.

Adatszerkezetek dokumentációja és tervezési megfontolások:

OO programozás és tervezési megfontolások:

Az objektumorientált programozást (OO) az alábbi előnyök miatt választottam:

Átláthatóság Az osztályok segítségével a program elkülönülő egységekre bontható amik lehetővé teszi a gyors és precíz karbantartást és hibakezelést.

Egységes kezelés:

Az öröklés és a heterogén kollekció megléte segít hogy könnyen bővíthető legyen a program például egy új hosszú esemény osztállyal

Karban tarthatóság:

Az osztályok függetlenek egymástól ha valamelyik adattagjuk változik akkor nem teszik tönkre az egész program működését

Tervezési döntéseim:

Minden osztály egyetlen felelőséggel rendelkezik pl: Ellenorzo Csak az adat validációért felel

Polimorfizmus:

Virtuális függvények felül írják az aalaposztály fv-ét.

String Osztály

Paraméterek:

pData (char*) A karaktereket tároló dinamikus tömb

len(size_t) A karakterlánc hossza (a lezáró \0 nélkül).

A függvény arra szolgál hogy ne keljen beépített sztringet használnom a programba az ustring laboron már foglalkoztunk vele én azokat a függvényeket mondom el amit én most irtam hozzá

bool operator==(const String& other) const;

Két sztring tartalmát hasonlítja össze .

bool tartalmaz(const String& reszlet) const;

Megvizsgálja hogy az adott sztring tartalmazza e az adott részletet

void beolvas(std::istream& is);

Beolvas karaktereket egy std::istream-ből (pl. std::cin) amíg \n-t nem talál.

Esemény osztály(alaposztály):

String datum;

String idopont;

String nev;

String helyszin;

String megjegyzes;

Ezek az adatok csak ebben az osztályban vagy az osztály leszármazottaiban érhatók el mivel protectedek.

Esemeny(const String& datum, const String& idopont, const String& nev, const String& helyszin, const String& megjegyzes);

Minden adattag kezdőértékét inicializálja és létrehoz egy esemény objektumot a megadott adatokkal.

const String& getDatum() const;

const String& getIdopont() const;

const String& getNev() const;

const String& getHelyszin() const;

const String& getMegjegyzes() const;

Ezek a függvények csak olvasható elérést biztosítanak az osztályt kívülről elérni akaró függvényeknek.

void setDatum(const String& ujDatum);

void setIdopont(const String& ujIdopont);

void setNev(const String& ujNev);

void setHelyszin(const String& ujHelyszin);

void setMegjegyzes(const String& ujMegjegyzes);

Megváltoztatja a megfelelő adattagok értékét.

virtual void kiir() const;

Kiírja az összes tagváltozót. Mivel virtuális ezért a leszármazott osztályok változtathatnak a működésén

virtual ~Esemeny()

Virtuális destructor amely biztosítja hogy a leszármazott destruktora is kellő képpen hívódjanak meg az objektum törlésekor.

visszatérőEsemény osztály(leszármazott osztály):

String gyakorisag;

Ez az adat csak ebben az osztályban érhető el mivel privát. Az esemény gyakoriságát tárolja(havi/heti/napi).

VisszateroEsemeny(const String& datum, const String& idopont, const String& nev, const String& helyszin, const String& megjegyzes, const String& gyakorisag);

Az ősoosztály (Esemeny) adattagjai + a gyakorisag.

Létrehoz egy VisszateroEsemeny objektumot

const String& getGyakorisag() const;

Ez a függvény csak olvasható elérést biztosít az osztályt kívülről elérni akaró függvényeknek.

void setGyakorisag(const String& ujGyakorisag);

Megváltoztatja a adattag értékét.

void kiir() const override;

Kiírja az összes tagváltozót az alap kiir függvényt abban változtatja hogy kiírja a gyakoriságot is

~VisszateroEsemeny();

Destruktor ami meg hívódik az objektum törlésekor.

FontosEsemeny osztály(leszármazott osztály):

prioritas: String

Az esemény prioritását tárolja (magas/alacsony) private tehát csak az osztályon belül érhető el.

**FontosEsemeny(const String& datum, const String& idopont, const String& nev,
const String& helyszin, const String& megjegyzes, const String& prioritas);**

Konstruktor

Létrehoz egy FontosEsemeny objektumot

Az őssosztály adattagjai + a prioritas.

const String& getPrioritas() const;

void setPrioritas(const String& ujPrioritas);

Azonos a VisszateroEsemeny getter/setter logikájával.

void kiir() const override;

Kiírja az esemény adatait + a prioritást.

~FontosEsemeny();

Destruktor ami meg hívódik az objektum törlésekor.

OnlineEsemeny osztály(leszármazott osztály):

url: String

Az esemény linkjét tárolja private tehát csak az osztályon belül érhető el.

**OnlineEsemeny(const String& datum, const String& idopont, const String& nev,
const String& helyszin, const String& megjegyzes, const String& url);**

Konstruktor

Az őssosztály (Esemeny) adattagjai + url.

Létrehoz egy OnlineEsemeny objektumot

const String& getUrl() const;

void setUrl(const String& uUrl);

Azonos a VisszateroEsemeny getter/setter logikájával.

void kiir() const override;

Kiírja az esemény adatait + az url.

~OnlineEsemeny();

Destruktor

Destruktor ami meg hívódik az objektum törlésekor.

Ellenorzo osztály:

ervenyesDatum(const String& datum)

ellenőrzi hogy pontosan 10 karakter e a hossz

érvényes számok vannak e az év/hónap/nap helyén

Hónap(1-12) és nap (1-31) tartomány.

ervenyesIdopont(const String& idopont)

5 pontos e a karakter hossz

Óra (0-23) és perc (0-59) tartomány

ervenyesNev(const String& nev)

Ellenőrzi hogy nem üres a sztring

csakSzamjegyek(const String& str);

Ellenőrzi hogy a sztring csak számjegyeket tartalmaz

ervenyesMenuPont(const String& input, int& menuPont);

A bemenet csak számjegyeket tartalmaz meghívja a csak Szamjegyek fv-t

Majd megnézi hogy a szám 1 és 7 közé esik e.

EsemenyKezelo osztály

Az Esemenykezelo osztály egy dinamikusan méretezett tömböt (Esemeny**) tárol, ami külön böző típusú eseményeket tárol

Adattagjai:

esemenyek: Esemeny** Dinamikus tomb események pointereit tárolja

meret:size_t tomb aktuális elemszáma

kapacitas:size_t A tömb maximális kapacitása(automatikusan duplázódik ha kell)

void bovitKapacitast();

Ez a függvény felelős a tároló dinamikus tömb(események) kapacitásának megduplázásáért amikor megtelik.(biztonságos memória kezelés)

EsemenyKezelo(size_t kezdKapacitas):

Paraméter: kezdKapacitas - A tömb kezdeti mérete (alapértelmezett 10)

Megadja a kezdő értékeit a tömbnekpldál a kezdő kapacitást

~EsemenyKezelo()

Felszabadítja az összes eseményt és a tömböt

beleteszEsemeny(Esemeny* e)

Paraméter: e - Hozzáadandó esemény pointer

Hozzáad egy eseményt a tömbhöz ha kell bővíti a kapacitást

hozzaadEsemeny()

Bekéri ma felhasználótól az új esemény adattagjait.Automatikusan deklarálja a típust

modositEsemeny()

Bekéri a dátumot és a nevet majd ez alapján megkeresi az eseményt majd lehetőséget nyújt a felhasználó számára az adattagok módosítására

void torolEsemeny()

Dátum és név alapján keres és töröl egy eseményt, a helyére nullptr-t tesz.

void listazEsemenyek() const

Kiírja az összes esemény adatait formázottan, az egyes események kiir() függvényét meghívva.

void keresEsemeny() const

Név vagy névrészlet alapján keres, és kiírja a találatokat.

Esemeny getEsemenyek()**

(Főleg a fájlkezelésben használok) visszaadja a tömböt.

int getMeret() const

Lekérdezi, hogy hány esemény van a tömbben.

Visszatérés: Az aktuális elemszám

FajlKezelo osztály:

mentesFajlba(Esemeny esemenyek, int meret)**

Paraméterek:

esemenyek: Az eseményeket tartalmazó tömb

meret: A tömb mérete

Működése:

A függvény arra szolgál hogy az eseményeket CSV formátumban mentse.

Bekéri a felhasználótól a fájlnevet majd megnyitja a fájlt végig megy az esemény tömbön alapeseményeket menti majd a különböző eseményeket is hozzáfűzi bezárja a fájlt és vissza jelzést ad.

Kiírás formátuma: dátum;időpont;név;helyszín;megjegyzés;[típus_specifikus_adat]

beolvasFajlbol(char* fajlNev)

Paraméter:

fajlNev: A beolvasandó fájl neve

Visszatérési érték:

Dinamikusan foglalt eseménytömb vagy null pointer

Működése:

A függvény arra szolgál hogy betöltse txt formátumból az adatokat amik ; vesszővel vannak elválasztva.

Megnyitja a fájlt majd soronként beolvassa az adatokat utána feldarabolja ; -ként az adatokat ellenőrzi a datok érvényességét következő lépésben létrehozza a megfelelő típusú eseményeket ha szükséges dinamikusan bővíti a tömböt majd bezárja a fájlt és visszaszadja a tömböt

Felhasznalo osztály:

menu(EsemenyKezelo& kezelo);

Paraméter:

kezelo: Az EsemenyKezelo osztály referenciája, amelyen keresztül az események kezelése történik.

Működése:

A függvény feladata a főmenü megjelenítése és kezelése.

Az egyes menüpontokkal kitudja válsztani a felhasználó hogy melyik feladatot hajtsa végre a program a program ellenőrzi a hibás bemeneteket.

main.cpp:

Ez a program belépési pontja, amely felelős az alkalmazás működésének elindításáért. Az események adatainak beolvasása után a felhasználói menüt jeleníti meg, amely lehetővé teszi a felhasználó számára az események kezelését. A program objektumorientált megközelítést követ, és integrálja az összes főbb osztályt.

Források:

https://en.cppreference.com/w/cpp/language/dynamic_cast (auto*és a dynamic_cast hoz)

geeksforgeeks egyes oldalai

https://www.geeksforgeeks.org/dynamic-_cast-in-cpp/

<https://www.geeksforgeeks.org/file-handling-c-classes/>

<https://www.geeksforgeeks.org/cin-get-in-c-with-examples/>

Az előadások anyaga és a laborok feladatai

